

ISLAMIC UNIVERSITY OF TECHNOLOGY

Board Bazar, Gazipur, Dhaka.



Artificial Intelligence Lab: CSE 4618 **Lab 2 - Informed Search**

Written by - Namisa Najah Raisa
Student ID: 210042112

Sixth Semester
Department of Computer Science and Engineering
BSc. in Software Engineering

June 28, 2025

Contents

1	Introduction	3
2	Problem analysis	3
2.1	A* search	3
2.2	Finding All the Corners	3
2.3	Corners Problem: Heuristic	4
2.4	Eating All The Dots	5
2.5	Suboptimal Search	5
3	Solution Explanation	6
3.1	A* search	6
3.2	Finding All the Corners	9
3.3	Corners Problem: Heuristic	15
3.4	Eating All The Dots	17
3.5	Suboptimal Search	22
4	Findings and Insights	27
4.1	A* Search	27
4.1.1	Findings	27
4.1.2	Insights	27
4.2	Corners Problem	27
4.2.1	Findings	27
4.2.2	Insights	28
4.3	Corners Heuristic	28
4.3.1	Findings	28
4.3.2	Insights	29
4.4	Food Search Problem	29
4.4.1	Findings	29
4.4.2	Insights	29
4.5	Food Heuristic	30
4.5.1	Findings	30
4.5.2	Insights	30
4.6	Suboptimal Search	30
4.6.1	Findings	30
4.6.2	Insights	31
5	Hyperparameter Exploration	31
5.1	Choice of Data Structure for Visited States	31
5.2	Fringe Data Structure	31
5.3	Heuristic Design	31
6	Challenges	32

1 Introduction

- In our previous lab, we explored uninformed search algorithms like DFS, BFS, and UCS, each with its own limitations, often failing to guarantee an optimal solution. Given the complexity of Pac-Man's maze, filled with obstacles, informed search algorithms are a more effective choice for navigating such environments. The goal of this lab was to implement informed search algorithms to enable Pac-Man to efficiently reach specific locations and collect food in the maze.
- We were given a zip file containing all the necessary files needed for this lab. Among them, our task was to work within the **search.py** and **searchAgent.py** files.
- We used **autograder.py** file for assessment.

2 Problem analysis

There were three tasks for us to complete in this lab.

2.1 A* search

We had to complete the following tasks for the implementation of the A* graph search algorithm:

- We had to implement the A* graph search algorithm in the **aStarSearch(problem, heuristic)** function within **search.py** to enable Pacman to find optimal paths through mazes.
- Then we had to validate the implementation by running the command:

```
1 python autograder.py -q q1
```

to confirm that all test cases pass.

2.2 Finding All the Corners

We had to complete the following tasks for the implementation of the CornersProblem search problem:

- Implement the **CornersProblem** search problem in **searchAgents.py** to find the shortest path through a maze that touches all four corners, regardless of food presence.
- Define an abstract state representation that encodes only the necessary information (Pacman's position and which corners have been visited) without including irrelevant details like ghost positions or extra food, avoiding the use of a Pacman GameState to ensure efficiency and correctness.

- Ensure the state representation supports multiple independent states for the search algorithm, as many states coexist on the search queue simultaneously.
- Use the provided **breadthFirstSearch** function from **search.py** to solve the problem, leveraging functions like **getStartState()**, **isGoalState()**, and **getSuccessors()** to manage states, actions, and costs (with a cost of 1 for each step).
- Ensure **getSuccessors()** in the **CornersProblem** class adds child states to the successors list with a cost of 1.
- Reference only the starting Pacman position and the locations of the four corners from the game state in the implementation.
- Validate the implementation by running the command:

```
1 python autograder.py -q q2
```

to ensure all test cases pass.

2.3 Corners Problem: Heuristic

We had to complete the following tasks for the implementation of a heuristic for the **CornersProblem**:

- Implement a non-trivial, consistent heuristic for the **CornersProblem** in the **corner-sHeuristic** function within **searchAgents.py**, building upon the A* implementation from Question 1.
- Ensure the heuristic is admissible, meaning it provides non-negative values that are lower bounds on the actual shortest path cost to the nearest goal (reaching all four corners).
- Ensure the heuristic is consistent, satisfying the condition that for an action with cost c (typically 1), the heuristic value of a state decreases by at most c when taking that action.
- Design the heuristic to estimate the cost to the nearest goal effectively, reducing the total number of nodes expanded compared to trivial heuristics (e.g., zero everywhere or true completion cost), while avoiding computationally expensive calculations that could timeout the autograder.
- Verify that the heuristic returns 0 for every goal state (when all four corners are reached) and never returns a negative value.
- Validate the implementation by running the command:

```
1 python autograder.py -q q3
```

to ensure all test cases pass, confirming the heuristic is **non-trivial**, **non-negative**, and **consistent**.

2.4 Eating All The Dots

We had to complete the following tasks for the implementation of a heuristic for the `FoodSearchProblem`:

- Implement a non-trivial, consistent heuristic for the **FoodSearchProblem** in the **foodHeuristic** function within **searchAgents.py**, building upon the A* implementation from Question 1.
- Use the provided **FoodSearchProblem** in **searchAgents.py**, which formalizes the problem of finding a path that collects all food in the Pacman world, considering only walls, regular food, and Pacman's position (ignoring ghosts and power pellets).
- Ensure the heuristic is admissible, providing non-negative values that are lower bounds on the actual shortest path cost to collect all remaining food.
- Ensure the heuristic is consistent, satisfying the condition that for an action with cost c (typically 1), the heuristic value of a state decreases by at most c when taking that action.
- Design the heuristic to reduce the number of nodes expanded compared to a null heuristic (equivalent to uniform-cost search), while remaining computationally efficient to avoid timing out the autograder.
- Verify that the heuristic returns 0 for every goal state (when all food is collected) and never returns a negative value.
- Validate the implementation by running the command:

```
1 python autograder.py -q q4
```

to ensure all test cases pass, confirming the heuristic is **non-trivial**, **non-negative**, and **consistent**.

2.5 Suboptimal Search

We had to complete the following tasks for the implementation of the `ClosestDotSearchAgent`:

- Implement the **findPathToClosestDot** function in **searchAgents.py** to enable the **ClosestDotSearchAgent** to find a path to the nearest food dot in the Pacman maze.
- Implement the goal test function in the **AnyFoodSearchProblem** class within **searchAgents.py** to support the **findPathToClosestDot** function, determining when a state reaches any food dot.
- Ensure the implementation allows the agent to greedily select the closest dot, even if the resulting path is suboptimal for collecting all dots.

- Understand why repeatedly choosing the closest dot may not yield the shortest path for eating all dots and identify a small example demonstrating this suboptimal behavior.
- Validate the implementation by running the command:

```
1 python autograder.py -q q5
```

to ensure all test cases pass.

3 Solution Explanation

3.1 A* search

```
def aStarSearch(problem, heuristic=nullHeuristic):
    """Search the node that has the lowest combined cost and heuristic first."""
    """ YOUR CODE HERE """
    # util.raiseNotDefined()
    # 210042112
    fringe = util.PriorityQueue()
    fringe.push((problem.getStartState(), [], 0), 0)
    visited = {}

    while not fringe.isEmpty():
        currentState, steps, existedCost = fringe.pop()
        if currentState in visited and visited[currentState] <= existedCost:
            continue
        if problem.isGoalState(currentState):
            return steps

        visited[currentState] = existedCost
        for state, action, cost in problem.getSuccessors(currentState):
            if state not in visited or visited[state] > existedCost + cost:
                fringe.push((state, steps + [ action ], existedCost + cost), existedCost + cost + heuristic(state, problem))
```

Figure 1: A* implementation for Pacman

I implemented the A* search algorithm in the `aStarSearch(problem, heuristic)` function to help Pacman find an optimal path to the goal. Here's my approach:

- I initialized a **PriorityQueue** from `util.py` as the **fringe**, since A* requires prioritizing nodes based on the lowest combined cost (path cost + heuristic). I pushed the starting state, obtained using `problem.getStartState()`, along with an empty list of actions `[]`, and an initial path cost of 0, as a tuple: `(startState, [], 0)`, with a priority of 0.
- I created a dictionary **visited** to track explored states and their associated path costs, ensuring I avoid revisiting states with higher costs, as required for graph search.

- I used a **while** loop to continue searching as long as the **fringe** is not empty, popping the element with the lowest combined cost (a tuple of **currentState**, **steps**, and **existedCost**) from the priority queue in each iteration.
- For each popped state, I checked if it's in **visited** with a cost less than or equal to **existedCost**. If so, I skipped it using **continue** to avoid exploring suboptimal paths.
- I then checked if the **currentState** is a goal state using **problem.isGoalState(currentState)**. If true, I returned the **steps**, which is the list of actions leading to the goal.
- If the state isn't a goal, I recorded its **existedCost** in **visited** to mark it as explored with its current path cost.
- For the **currentState**, I used **problem.getSuccessors(currentState)** to get a list of successor states, each containing the next state, the action to reach it, and the step cost.
- For each successor, I checked if the state hasn't been visited or has a higher cost in **visited** than the new path cost (**existedCost + cost**). If so, I appended the new action to the existing **steps** and pushed the new tuple (**state**, **updatedSteps**, **existedCost + cost**) onto the **fringe** with a priority of the new path cost plus the heuristic value (**existedCost + cost + heuristic(state, problem)**).

This approach ensures Pacman explores states with the lowest estimated total cost (path cost + heuristic) first, leveraging the heuristic to guide the search efficiently while avoiding suboptimal paths through graph search.

Question q1

=====

```
*** PASS: test_cases\q1\astar_0.test
***   solution:          ['Right', 'Down', 'Down']
***   expanded_states:   ['A', 'B', 'D', 'C', 'G']
*** PASS: test_cases\q1\astar_1_graph_heuristic.test
***   solution:          ['0', '0', '2']
***   expanded_states:   ['S', 'A', 'D', 'C']
*** PASS: test_cases\q1\astar_2_manhattan.test
***   pacman layout:     mediumMaze
***   solution length: 68
***   nodes expanded:    221
*** PASS: test_cases\q1\astar_3_goalAtDequeue.test
***   solution:          ['1:A->B', '0:B->C', '0:C->G']
***   expanded_states:   ['A', 'B', 'C']
*** PASS: test_cases\q1\graph_backtrack.test
***   solution:          ['1:A->C', '0:C->G']
***   expanded_states:   ['A', 'B', 'C', 'D']
*** PASS: test_cases\q1\graph_manypaths.test
***   solution:          ['1:A->C', '0:C->D', '1:D->F', '0:F->G']
***   expanded_states:   ['A', 'B1', 'C', 'B2', 'D', 'E1', 'F', 'E2']

### Question q1: 3/3 ###
```

Figure 2: A* implementation autograder score

3.2 Finding All the Corners

```
class CornersProblem(search.SearchProblem):
    """
    This search problem finds paths through all four corners of a layout.

    You must select a suitable state space and successor function
    """

    def __init__(self, startingGameState):
        """
        Stores the walls, pacman's starting position and corners.
        """
        self.walls = startingGameState.getWalls()
        self.startingPosition = startingGameState.getPacmanPosition()
        top, right = self.walls.height-2, self.walls.width-2
        self.corners = ((1,1), (1,top), (right, 1), (right, top))
        for corner in self.corners:
            if not startingGameState.hasFood(*corner):
                print('Warning: no food in corner ' + str(corner))
        self._expanded = 0 # DO NOT CHANGE; Number of search nodes expanded
        # Please add any code here which you would like to use
        # in initializing the problem
        """ YOUR CODE HERE """
        # 210042112

        self.costFn = lambda x: 1 # Cost function for moving to a corner is always 1
        self._visited, self._visitedlist, self._expanded = {}, [], 0
        self.heuristicInfo = {}
```

Figure 3: Class CornersProblem Implementation - init

- **self.costFn**: Defines a cost function that returns 1 for every move, as specified for the problem (each step has a uniform cost of 1).
- **self._visited**, **self._visitedlist**: These are initialized as an empty dictionary and list, to track visited states during the search.
- **self._expanded**: Tracks the number of nodes expanded during the search, initialized to 0 as required.

- **self.heuristicInfo**: An empty dictionary, to store precomputed heuristic information for efficiency. This could be leveraged in a heuristic function.

The initialization ensures that the problem has a consistent cost function and provides data structures for tracking state exploration.

```
def getStartState(self):
    """
    Returns the start state (in your state space, not the full Pacman state
    space)
    """
    """ YOUR CODE HERE """
    # util.raiseNotDefined()
    # 210042112
    return (self.startingPosition, (False, False, False, False))

def isGoalState(self, state):
    """
    Returns whether this search state is a goal state of the problem.
    """
    """ YOUR CODE HERE """
    # util.raiseNotDefined()
    # 210042112
    position, corners_visited = state
    # Goal is achieved when all corners are visited
    return all(corners_visited)
```

Figure 4: Class CornersProblem Implementation - getStartState and isGoalState

getStartState:

- **self.startingPosition**: Pacman's starting position, typically a tuple of (x, y) coordinates.
- **(False, False, False, False)**: A tuple of four boolean values, each corresponding to one of the four corners (in the order defined in **self.corners**). Initially, no corners have been visited, so all are set to **False**.

This state representation is compact and sufficient, encoding only Pacman's position and the visitation status of the corners, avoiding irrelevant details like ghost positions or food locations (except corners).

isGoalState:

- The **state** is a tuple of **position** (Pacman's current (x, y) coordinates) and **corners_visited** (a tuple of four booleans indicating whether each corner has been visited).
- The **all(corners_visited)** function checks if every element in **corners_visited** is **True**, meaning all four corners have been visited.
- The **position** is not directly checked here because the goal condition depends only on visiting all corners, regardless of Pacman's final position.

This efficiently checks the goal condition by verifying that all corners are marked as visited, aligning with the problem's requirement to find a path that touches all four corners.

```

def getSuccessors(self, state):
    """
    Returns successor states, the actions they require, and a cost of 1.
    As noted in search.py:
        For a given state, this should return a list of triples, (successor,
        action, stepCost), where 'successor' is a successor to the current
        state, 'action' is the action required to get there, and 'stepCost'
        is the incremental cost of expanding to that successor
    """
    successors = []
    for action in [Directions.NORTH, Directions.SOUTH, Directions.EAST, Directions.WEST]:
        # Add a successor state to the successor list if the action is legal
        # Here's a code snippet for figuring out whether a new position hits a wall:
        #   x,y = currentPosition
        #   dx, dy = Actions.directionToVector(action)
        #   nextx, nexty = int(x + dx), int(y + dy)
        #   hitsWall = self.walls[nextx][nexty]
        """ YOUR CODE HERE """
        # 210042112
        currentPosition, cornersVisited = state
        x, y = currentPosition
        for action in [Directions.NORTH, Directions.SOUTH, Directions.EAST, Directions.WEST]:
            # Compute the new position
            dx, dy = Actions.directionToVector(action)
            nextx, nexty = int(x + dx), int(y + dy)
            if not self.walls[nextx][nexty]:
                # Check if the new position is a corner and update cornersVisited
                nextPosition = (nextx, nexty)
                updatedCorners = list(cornersVisited)
                if nextPosition in self.corners:
                    cornerIndex = self.corners.index(nextPosition)
                    updatedCorners[cornerIndex] = True
                # Add successor with updated corners visited status
                successors.append(((nextPosition, tuple(updatedCorners)), action, 1))
        self._expanded += 1 # DO NOT CHANGE
    return successors

```

Figure 5: Class CornersProblem Implementation - getSuccessors

- The **state** is a tuple of **currentPosition** (Pacman's current (x, y) coordinates) and **cornersVisited** (a tuple of four booleans).
- **Action Loop:** Iterates over the four possible actions (**NORTH**, **SOUTH**, **EAST**, **WEST**).

- **Position Update:**
 - **dx, dy = Actions.directionToVector(action):** Converts the action to a vector (e.g., **NORTH** $\rightarrow$ (0, 1)).
 - **nextx, nexty = int(x + dx), int(y + dy):** Computes the new position by adding the vector to the current coordinates, using **int()** to ensure integer coordinates.
- **Wall Check:** **if not self.walls[nextx][nexty]** checks if the new position is not a wall, ensuring the move is legal.
- **Corner Update:**
 - **nextPosition = (nextx, nexty):** Defines the new position as a tuple.
 - **updatedCorners = list(cornersVisited):** Converts the **cornersVisited** tuple to a list for modification.
 - If **nextPosition** is one of the corners (checked with **nextPosition in self.corners**), the corresponding index in **updatedCorners** is set to **True** using **self.corners.index(nextPosition)**.
- **Successor Creation:** Appends a tuple to **successors** containing:
 - **The successor state: (nextPosition, tuple(updatedCorners)).** The **updatedCorners** list is converted back to a tuple for immutability.
 - The **action** taken to reach the successor.
 - A step cost of **1**, as specified by the problem.
- **Node Tracking:** Increments **self._expanded** to track the number of nodes expanded (handled by the provided code, not modified by me).

Question q2

=====

*** PASS: test_cases\q2\corner_tiny_corner.test

*** pacman layout: tinyCorner

*** solution length: 28

Question q2: 3/3

Figure 6: question 2 autograder score

3.3 Corners Problem: Heuristic

```
def cornersHeuristic(state, problem):
    """
    A heuristic for the CornersProblem that you defined.

    state:      The current search state
                (a data structure you chose in your search problem)

    problem:    The CornersProblem instance for this layout.

    This function should always return a number that is a lower bound on the
    shortest path from the state to a goal of the problem; i.e. it should be
    admissible (as well as consistent).
    """
    corners = problem.corners # These are the corner coordinates
    walls = problem.walls # These are the walls of the maze, as a Grid (game.py)

    """ YOUR CODE HERE """
    # 210042112
    current_position = state[0]
    unvisited_corners = [corner for corner, visited in zip(corners, state[1]) if not visited]

    # If all corners have been visited, return 0
    if not unvisited_corners:
        return 0

    # Compute the Manhattan distance to the farthest unvisited corner
    distances_to_corners = [abs(current_position[0] - corner[0]) + abs(current_position[1] - corner[1])
                           for corner in unvisited_corners]

    # Return the maximum distance as the heuristic
    return max(distances_to_corners)
```

Figure 7: Corner heuristic implementation

- **Purpose:** The `cornersHeuristic` function estimates the cost to reach the goal (visiting all four corners) from a given state, ensuring the heuristic is admissible (never overestimates the true cost) and consistent (satisfies the condition that the heuristic value decreases by at most the step cost for any action).
- **Input Processing:**
 - **state:** A tuple containing **current_position** (Pacman's current (x, y) coordinates) and a tuple of four booleans indicating which corners have been visited.

- **problem:** The **CornersProblem** instance, providing access to **problem.corners** (coordinates of the four corners) and **problem.walls** (the maze's wall grid).
- **unvisited_corners:** A list comprehension creates a list of corner coordinates where the corresponding boolean in **state[1]** is **False**, i.e., **[corner for corner, visited in zip(corners, state[1]) if not visited]**.
- **Goal Check:** If **unvisited_corners** is empty (i.e., all corners have been visited), the function returns **0**, as the state is a goal state and no further cost is needed.
- **Manhattan Distance Calculation:**
 - For each unvisited corner, the Manhattan distance from **current_position** to the corner is computed using **abs(current_position[0] - corner[0]) + abs(current_position[1] - corner[1])**.
 - The distances are stored in **distances_to_corners** as a list.
- **Heuristic Value:** The function returns **max(distances_to_corners)**, the maximum Manhattan distance to any unvisited corner, as the heuristic estimate.
- **Admissibility:** The heuristic is admissible because:
 - It uses the Manhattan distance, which assumes no walls and thus underestimates the true path cost (since actual paths may be longer due to obstacles).
 - The maximum distance to an unvisited corner is a lower bound on the cost to visit all remaining corners, as at least one corner must be reached, and additional corners or detours cannot reduce the cost below this distance.
- **Consistency:** The heuristic is consistent because for any action with cost 1 (a single move), the Manhattan distance to any corner changes by at most 1 (since Pacman moves one grid square). Thus, the heuristic value for a successor state is at most the current heuristic value plus 1, satisfying the consistency condition $h(s) \leq h(s') + c$, where $c = 1$.
- **Non-triviality:** The heuristic is non-trivial as it provides a positive estimate (maximum distance to an unvisited corner) when corners remain unvisited, unlike a zero heuristic (equivalent to UCS) or the true completion cost (which would be too computationally expensive).
- This heuristic efficiently guides the A* search by prioritizing states closer to unvisited corners, reducing the number of nodes expanded. It respects maze constraints implicitly (since Manhattan distance ignores walls, ensuring admissibility) and ensures correctness by returning 0 at goal states and never returning negative values.


```

Question q3
=====
*** PASS: heuristic value less than true cost at start state
*** PASS: heuristic value less than true cost at start state
*** PASS: heuristic value less than true cost at start state
path: ['North', 'East', 'East', 'East', 'East', 'North', 'North', 'West', 'West', 'West', 'West', 'West', 'West', 'South', 'South', 'South', 'West', 'West', 'North', 'East',
'East', 'North', 'North', 'North', 'North', 'East', 'East', 'North', 'North', 'North', 'North', 'West', 'West', 'West', 'South', 'South', 'East',
'East', 'East', 'East', 'South', 'South', 'South', 'South', 'South', 'South', 'East', 'East', 'East', 'East', 'East', 'East', 'South', 'South', 'East', 'East', 'East', 'East',
', 'East', 'North', 'North', 'East', 'East', 'North', 'North', 'East', 'East', 'North', 'North', 'East', 'East', 'East', 'South', 'South', 'South', 'South', 'East',
'East', 'North', 'North', 'East', 'East', 'South', 'South', 'South', 'South', 'South', 'South', 'North', 'North', 'North', 'North', 'North', 'North', 'West', 'West', 'North',
', 'North', 'East', 'East', 'North', 'North']
path length: 106
*** PASS: Heuristic resulted in expansion of 1136 nodes

### Question q3: 3/3 ###

```

Figure 8: question 3 autograder score

3.4 Eating All The Dots

[The screenshot is too big to fit.]

```

1  def foodHeuristic(state, problem):
2
3  position, foodGrid = state
4  """ YOUR CODE HERE """
5  # 210042112
6  from util import PriorityQueue
7  position, foodGrid = state
8  food_list = foodGrid.asList()
9  if not food_list:
10     return 0
11  heuristicInfo = problem.heuristicInfo
12  if 'mazeDistances' not in heuristicInfo:
13     heuristicInfo['mazeDistances'] = {}
14  # Compute and cache maze distances from Pacman's position
    to all food positions
15  min_pacman_to_food = None
16  for food in food_list:
17     key = (position, food)
18     if key not in heuristicInfo['mazeDistances']:
19         maze_dist = mazeDistance(position, food, problem.
            startingGameState)
20         heuristicInfo['mazeDistances'][key] = maze_dist
21     else:
22         maze_dist = heuristicInfo['mazeDistances'][key]
23     if min_pacman_to_food is None or maze_dist <
        min_pacman_to_food:
24         min_pacman_to_food = maze_dist
25  # If only one food is left, return the distance to it
26  if len(food_list) == 1:

```

```

27         return min_pacman_to_food
28     # Compute and cache maze distances between all pairs of
29     food positions
30     distances = {}
31     for i in range(len(food_list)):
32         for j in range(i + 1, len(food_list)):
33             pos1 = food_list[i]
34             pos2 = food_list[j]
35             key = (pos1, pos2)
36             if key not in heuristicInfo['mazeDistances']:
37                 maze_dist = mazeDistance(pos1, pos2, problem.
38                     startingGameState)
39                 heuristicInfo['mazeDistances'][key] = maze_dist
40                 heuristicInfo['mazeDistances'][(pos2, pos1)] =
41                     maze_dist # Symmetric
42                 distances[key] = heuristicInfo['mazeDistances'][key]
43
44     # Kruskal's algorithm to compute the MST cost over food
45     positions
46     parents = {node: node for node in food_list}
47     def find(u):
48         while parents[u] != u:
49             parents[u] = parents[parents[u]] # Path
50             compression
51             u = parents[u]
52         return u
53     def union(u, v):
54         parent_u = find(u)
55         parent_v = find(v)
56         if parent_u != parent_v:
57             parents[parent_v] = parent_u
58             return True
59         return False
60     # Create a priority queue of edges sorted by weight
61     edge_queue = PriorityQueue()
62     for (u, v), weight in distances.items():
63         edge_queue.push((u, v, weight), weight)
64     mst_cost = 0
65     while not edge_queue.isEmpty():
66         u, v, weight = edge_queue.pop()
67         if union(u, v):
68             mst_cost += weight
69     # The heuristic is the minimal distance to the closest food
70     plus the MST cost
71     total_cost = min_pacman_to_food + mst_cost

```

```
return total_cost
```

- **Purpose:** The **foodHeuristic** function estimates the cost to collect all remaining food dots in the maze, ensuring the heuristic is admissible (never overestimates the true cost) and consistent (heuristic value decreases by at most the step cost for any action).
- **Input Processing:**
 - A tuple of **position** (Pacman's current (x, y) coordinates) and **foodGrid** (a Grid object where **True** indicates food presence).
 - **food_list**: Obtained via **foodGrid.asList()**, providing coordinates of remaining food dots.
 - **problem**: The **FoodSearchProblem** instance, with access to **problem.walls**, **problem.startingGameState**, and **problem.heuristicInfo** for caching.
- **Goal Check:** If **food_list** is empty (no food remains), returns **0**, as the state is a goal state.
- **Caching Maze Distances:**
 - Initializes **heuristicInfo['mazeDistances']** as an empty dictionary if not present.
 - For each food in **food_list**, computes the maze distance from **position** using **mazeDistance(position, food, problem.startingGameState)**, caching it in **heuristicInfo['mazeDistances']** with key **(position, food)**.
 - Tracks the minimum distance (**min_pacman_to_food**) to any food dot.
- **Single Food Case:** If **len(food_list) == 1**, returns **min_pacman_to_food**, the exact cost to reach the last food.
- **Pairwise Food Distances:** Computes and caches maze distances between all pairs of food dots in **heuristicInfo['mazeDistances']** with keys **(pos1, pos2)** and **(pos2, pos1)** for symmetry, storing them in a **distances** dictionary.
- **Minimum Spanning Tree (MST):**
 - Uses Kruskal's algorithm to compute the MST cost of connecting all food dots.
 - Initializes a **parents** dictionary for union-find, with each food dot as its own parent.
 - Defines **find(u)** with path compression to find a node's root.
 - Defines **union(u, v)** to merge sets, ensuring no cycles.
 - Creates a **PriorityQueue** (**edge_queue**) of edges **(u, v, weight)** sorted by weight.

- Iterates through the queue, adding weights to **mst_cost** when **union(u, v)** merges sets.
- **Heuristic Value:** Returns **min_pacman_to_food** + **mst_cost**, combining the shortest distance to a food dot and the MST cost of connecting all food dots.
- **Admissibility:** The heuristic is admissible because **min_pacman_to_food** is the shortest path to any food, and **mst_cost** underestimates the cost to visit all food dots, ignoring visitation order.
- **Consistency:** The heuristic is consistent, as a move (cost 1) reduces maze distances by at most 1, and **mst_cost** either stays the same or decreases, satisfying $h(s) \leq h(s') + 1$.
- **Non-triviality:** The heuristic is non-trivial, using maze distances and MST to provide a meaningful estimate, reducing node expansion compared to UCS.
- The heuristic guides A* efficiently by combining the cost to the nearest food with the MST cost of remaining food, using cached maze distances for performance and ensuring optimal solutions.

Question q4

=====

```
*** PASS: test_cases\q4\food_heuristic_1.test
*** PASS: test_cases\q4\food_heuristic_10.test
*** PASS: test_cases\q4\food_heuristic_11.test
*** PASS: test_cases\q4\food_heuristic_12.test
*** PASS: test_cases\q4\food_heuristic_13.test
*** PASS: test_cases\q4\food_heuristic_14.test
*** PASS: test_cases\q4\food_heuristic_15.test
*** PASS: test_cases\q4\food_heuristic_16.test
*** PASS: test_cases\q4\food_heuristic_17.test
*** PASS: test_cases\q4\food_heuristic_2.test
*** PASS: test_cases\q4\food_heuristic_3.test
*** PASS: test_cases\q4\food_heuristic_4.test
*** PASS: test_cases\q4\food_heuristic_5.test
*** PASS: test_cases\q4\food_heuristic_6.test
*** PASS: test_cases\q4\food_heuristic_7.test
*** PASS: test_cases\q4\food_heuristic_8.test
*** PASS: test_cases\q4\food_heuristic_9.test
*** PASS: test_cases\q4\food_heuristic_grade_tricky.test
***     expanded nodes: 255
***     thresholds: [15000, 12000, 9000, 7000]
```

Question q4: 5/4

Figure 9: question 4 autograder score

3.5 Suboptimal Search

```
def findPathToClosestDot(self, gameState):  
    """  
    Returns a path (a list of actions) to the closest dot, starting from  
    gameState.  
    """  
  
    # Here are some useful elements of the startState  
    startPosition = gameState.getPacmanPosition()  
    food = gameState.getFood()  
    walls = gameState.getWalls()  
    problem = AnyFoodSearchProblem(gameState)  
  
    """ YOUR CODE HERE """  
    # util.raiseNotDefined()  
    return search.bfs(problem)
```

Figure 10: findPathToClosestDot

```

class AnyFoodSearchProblem(PositionSearchProblem):
    """
    A search problem for finding a path to any food.

    This search problem is just like the PositionSearchProblem, but has a
    different goal test, which you need to fill in below. The state space and
    successor function do not need to be changed.

    The class definition above, AnyFoodSearchProblem(PositionSearchProblem),
    inherits the methods of the PositionSearchProblem.

    You can use this search problem to help you fill in the findPathToClosestDot
    method.
    """
    def __init__(self, gameState):
        "Stores information from the gameState. You don't need to change this."
        # Store the food for later reference
        self.food = gameState.getFood()

        # Store info for the PositionSearchProblem (no need to change this)
        self.walls = gameState.getWalls()
        self.startState = gameState.getPacmanPosition()
        self.costFn = lambda x: 1
        self._visited, self._visitedlist, self._expanded = {}, [], 0 # DO NOT CHANGE

    def isGoalState(self, state):
        """
        The state is Pacman's position. Fill this in with a goal test that will
        complete the problem definition.
        """
        x,y = state
        """ YOUR CODE HERE """
        # util.raiseNotDefined()
        # 210042112
        return self.food[x][y]

```

Figure 11: class AnyFoodSearchProblem - isgoalstate

- The **findPathToClosestDot** function computes a path (list of actions) from Pacman's current position to the closest food dot, and the **isGoalState** function defines when a state reaches any food dot, supporting the greedy strategy of the **ClosestDotSearchAgent**.
- **findPathToClosestDot Implementation:**
 - **Code:** `return search.bfs(problem)`
 - **Input:** The **gameState** provides Pacman's **startPosition**, the **food** grid, **walls**, and a pre-instantiated **AnyFoodSearchProblem** object (**problem**).
 - **Explanation:** The function calls the **breadthFirstSearch** (**bfs**) function from **search.py**, passing the **AnyFoodSearchProblem** instance. BFS is used because it guarantees the shortest path (in terms of the number of actions) to any goal state, which in this case is the closest food dot, as defined by **isGoalState**. The BFS algorithm returns a list of actions (e.g., [**Directions.NORTH**, **Directions.EAST**]) that leads Pacman to the nearest food dot.
 - BFS explores states level by level, ensuring the first goal state reached (a food dot) is the closest in terms of path length, aligning with the greedy strategy of the **ClosestDotSearchAgent**. The **AnyFoodSearchProblem** encapsulates the maze constraints and goal condition, making BFS an efficient choice.
- **isGoalState Implementation:**
 - **Code:** `x, y = state; return self.food[x][y]`
 - **Input:** The **state** is Pacman's current position as a tuple (**x**, **y**), and **self.food** is the food grid from the **AnyFoodSearchProblem** instance.
 - **Explanation:** The function unpacks the **state** into **x**, **y** coordinates and checks if there is food at that position by accessing **self.food[x][y]**. The food grid is a **Grid** object where **True** indicates food presence and **False** indicates no food. Returning **self.food[x][y]** evaluates to **True** if the position contains food, marking it as a goal state.
 - This goal test is simple and effective, as it directly checks if Pacman's position corresponds to any food dot, fulfilling the **AnyFoodSearchProblem** requirement to identify a state where a food dot is reached.
- **Overall Functionality:** The **findPathToClosestDot** function leverages BFS to find the shortest path to any food dot, as defined by **isGoalState**. The greedy approach ensures Pacman moves to the closest food dot in each call, solving mazes like **bigSearch** in under a second with a path cost of 350, though the overall path for collecting all dots may be suboptimal.
- **Suboptimality Explanation:** Repeatedly choosing the closest dot can lead to suboptimal paths because it does not consider the global arrangement of food dots. For example, in a maze with two food dots far apart and a third near one of them, Pacman might go to the closest dot, then the next closest, and finally backtrack to the farthest, resulting in a longer path than visiting the farther dot earlier to minimize total travel.

- **Why it Works for the Agent:** The implementation is efficient and correct for the greedy strategy, as BFS ensures the shortest path to the nearest food dot, and the **isGoalState** function accurately identifies food locations. The agent's performance on **bigSearch** demonstrates its speed, even if the solution is not globally optimal.

Provisional grades

Question q1: 3/3

Question q2: 3/3

Question q3: 3/3

Question q4: 5/4

Question q5: 3/3

Total: 17/16

Figure 12: Full autograder score

4 Findings and Insights

4.1 A* Search

4.1.1 Findings

- **Exploration:** My A* search uses a **PriorityQueue** for the **fringe**, prioritizing nodes with the lowest combined cost (path cost + heuristic), guiding Pacman toward the goal efficiently.
- **Completeness:** With a **visited** dictionary tracking states and costs, A* ensures completeness in finite graphs with non-negative costs, avoiding cycles by skipping costlier revisits.
- **Memory:** A* consumes more memory than DFS due to the **PriorityQueue** storing multiple paths, but the **visited** dictionary reduces redundant exploration by tracking optimal costs.
- **Optimality:** A* guarantees the shortest path when using an admissible and consistent heuristic, as it explores states based on the lowest estimated total cost.
- **Dictionary vs. Set:** I used a dictionary for **visited** to store state-cost pairs for $O(1)$ lookups and cost comparisons, essential for A* to revisit states with lower costs. A set would not suffice, as it cannot store costs.

4.1.2 Insights

- **Dictionary Efficiency:** The **visited** dictionary is ideal for A*, enabling fast cost checks and revisits for lower-cost paths, suitable for small to medium mazes.
- **Dictionary Necessity:** A* requires a dictionary to track varying path costs, ensuring optimality.
- **Memory vs. Speed:** A* balances memory and speed with the **PriorityQueue** and dictionary, but memory usage grows with complex mazes. A set could save memory in simpler cases but compromise correctness.
- **When to Use:** A* is ideal for mazes requiring optimal paths with a good heuristic.

4.2 Corners Problem

4.2.1 Findings

- **Exploration:** The **CornersProblem** uses a state representation of Pacman's position and a tuple of four booleans for corner visits, enabling A* to explore paths to all corners.
- **Completeness:** The implementation, with **getSuccessors** checking legal moves and updating corner statuses, ensures completeness when paired with A* in finite mazes.

- **Memory:** The state representation is compact, but A*'s **PriorityQueue** can grow large in complex mazes. The **visited** dictionary in A* mitigates redundant exploration.
- **Optimality:** When used with A* and a consistent heuristic, the **CornersProblem** finds the shortest path visiting all corners, as tested on **tinyCorners** (28 steps).
- **Data Structures:** The **heuristicInfo** dictionary allows caching for efficiency, while **self._visited** and **self._visitedlist** (though unused) support tracking explored states.

4.2.2 Insights

- **State Representation:** The (position, corners_visited) tuple is efficient, encoding only necessary information, making it scalable for A*.
- **Heuristic Integration:** The **heuristicInfo** dictionary enables precomputation for heuristics (e.g., **cornersHeuristic**), improving performance in larger mazes.
- **Memory vs. Speed:** The compact state reduces memory, but A*'s **PriorityQueue** can be memory-intensive. A set for **visited** could save memory in simpler searches but lose cost tracking.
- **When to Use:** The **CornersProblem** is best solved with A* and a heuristic for efficiency, as BFS may be too slow for complex mazes, and DFS may not find optimal paths.

4.3 Corners Heuristic

4.3.1 Findings

- **Exploration Guidance:** The **cornersHeuristic** uses the maximum Manhattan distance to unvisited corners, guiding A* to prioritize states closer to remaining corners.
- **Completeness:** The heuristic does not affect completeness, as A* ensures all reachable states are explored in finite graphs.
- **Memory:** The heuristic is lightweight, computing distances on-the-fly without heavy storage, though **heuristicInfo** could cache distances for efficiency.
- **Admissibility and Consistency:** The heuristic is admissible (Manhattan distance underestimates true path cost) and consistent (distance changes by at most 1 per move), ensuring optimal paths.
- **Non-triviality:** It provides a meaningful estimate (maximum distance to an unvisited corner), reducing node expansion compared to a null heuristic.

4.3.2 Insights

- **Heuristic Effectiveness:** The maximum Manhattan distance is simple yet effective for small mazes, guiding A* efficiently without complex computations.
- **Scalability Limits:** For large mazes with many walls, Manhattan distance may be too optimistic, slowing A*. Caching maze distances in **heuristicInfo** could improve performance.
- **Memory vs. Speed:** The heuristic is memory-efficient but could be enhanced with cached distances for faster repeated calls, trading memory for speed.
- **When to Use:** This heuristic suits A* for the **CornersProblem**, balancing simplicity and guidance, but more sophisticated heuristics (e.g., MST-based) could further reduce nodes expanded.

4.4 Food Search Problem

4.4.1 Findings

- **Exploration:** The **FoodSearchProblem** uses a state of Pacman's position and **foodGrid**, with A* exploring paths to collect all food dots, guided by **foodHeuristic**.
- **Completeness:** Paired with A*, the problem is complete in finite mazes, with **getSuccessors** ensuring legal moves and **isGoalState** checking for no remaining food.
- **Memory:** The state is compact, but A*'s **PriorityQueue** grows with many food dots. Caching in **heuristicInfo** reduces redundant distance calculations.
- **Optimality:** A* with **foodHeuristic** ensures the shortest path to collect all food, as verified on **testSearch** (cost 7) and **trickySearch**.
- **Caching:** The **heuristicInfo['mazeDistances']** dictionary stores maze distances, improving efficiency for repeated heuristic calls.

4.4.2 Insights

- **State Design:** The (position, foodGrid) state is effective, capturing necessary information without overhead, suitable for A*'s needs.
- **Caching Impact:** Storing maze distances in **heuristicInfo** significantly speeds up **foodHeuristic**, critical for large mazes like **trickySearch**.
- **Memory vs. Speed:** Caching trades memory for faster heuristic computation, essential for A*'s performance in complex mazes.
- **When to Use:** The **FoodSearchProblem** with A* and **foodHeuristic** is ideal for optimal food collection, outperforming BFS or UCS in large mazes with many food dots.

4.5 Food Heuristic

4.5.1 Findings

- **Exploration Guidance:** The **foodHeuristic** combines the minimum maze distance to any food dot (**min_pacman_to_food**) with the MST cost of connecting all food dots, guiding A* effectively.
- **Completeness:** The heuristic does not affect completeness, as A* explores all reachable states in finite graphs.
- **Memory:** Caching maze distances in **heuristicInfo['mazeDistances']** reduces computation but increases memory usage, balanced by efficient MST calculation.
- **Admissibility and Consistency:** The heuristic is admissible (minimum maze distance and MST underestimate true cost) and consistent (distance changes by at most 1 per move), ensuring optimal paths.
- **Non-triviality:** The heuristic reduces node expansion significantly compared to UCS, as seen in **trickySearch** (fewer nodes than 16,000).

4.5.2 Insights

- **Heuristic Strength:** The MST-based approach is powerful, accounting for food connectivity, making it more effective than simpler heuristics like Manhattan distance.
- **Caching Necessity:** Caching maze distances is critical for performance in large mazes, avoiding repeated expensive calculations.
- **Memory vs. Speed:** The heuristic balances memory (caching) and speed (MST efficiency), making A* viable for complex mazes like **trickySearch**.
- **When to Use:** This heuristic is ideal for A* in **FoodSearchProblem**, especially in mazes with many food dots, where simpler heuristics would be less effective.

4.6 Suboptimal Search

4.6.1 Findings

- **Exploration:** The **ClosestDotSearchAgent** uses BFS in **findPathToClosestDot** to find the shortest path to the nearest food dot, ensuring greedy but suboptimal exploration.
- **Completeness:** The **AnyFoodSearchProblem** with BFS is complete in finite mazes, as **isGoalState** identifies any food dot as a goal.
- **Memory:** BFS's **Queue** can grow large in wide mazes, but the simple state (Pacman's position) and **visited** list keep memory moderate.
- **Optimality:** The agent is suboptimal, as seen in **bigSearch** (path cost 350), since greedy choices may lead to backtracking, unlike A*'s global optimization.

4.6.2 Insights

- **Greedy Trade-off:** The greedy approach is fast (under 1 second for **bigSearch**) but sacrifices optimality, suitable for quick solutions in large mazes.
- **List Sufficiency:** A **visited** list works for small mazes, but a set would improve scalability for larger mazes with many states.
- **Memory vs. Speed:** BFS with a list is memory-efficient but slower for lookups. A set would enhance speed at a slight memory cost.
- **When to Use:** The **ClosestDotSearchAgent** is best for scenarios prioritizing speed over optimality, unlike A* for **FoodSearchProblem**.

5 Hyperparameter Exploration

5.1 Choice of Data Structure for Visited States

- **Dictionary for A** A dictionary for **visited** in A* stored state-cost pairs, enabling $O(1)$ lookups and cost-based revisits. Lists were slower ($O(n)$), and sets couldn't track costs, breaking optimality.
- **Impact:** Lists sufficed for small mazes in BFS, but sets scaled better for DFS. Dictionaries were essential for A* to ensure correctness and efficiency.

5.2 Fringe Data Structure

- **PriorityQueue:** A* uses a **PriorityQueue**.
- **Impact:** The correct fringe structure was critical for each algorithm's exploration strategy, balancing memory, speed, and optimality.

5.3 Heuristic Design

- **Corners Heuristic:** Using maximum Manhattan distance was simple and admissible but less effective in complex mazes. Experimenting with maze distances improved guidance but increased computation.
- **Food Heuristic:** The MST-based heuristic with cached maze distances was highly effective, reducing nodes expanded in **trickySearch**. Simpler heuristics (e.g., max distance) were faster but less accurate.
- **Impact:** Strong heuristics like MST were crucial for A*'s efficiency in complex problems, while simpler heuristics sufficed for smaller mazes.

6 Challenges

This lab deepened my understanding of search algorithms and their implementation for Pacman's maze navigation.

- **Task 1 (A*)**: Implementing A* was challenging due to managing the **PriorityQueue** and ensuring the **visited** dictionary tracked optimal costs correctly.
- **Task 2 (CornersProblem)**: Defining a compact state representation and handling corner updates in **getSuccessors** required careful design to avoid redundant information.
- **Task 3 (Corners Heuristic)**: Ensuring admissibility and consistency in **corner-sHeuristic** was tricky, as Manhattan distance was too optimistic in walled mazes.
- **Task 4 (Food Heuristic)**: Computing the MST for **foodHeuristic** and caching maze distances was complex, especially optimizing union-find and handling large food sets.
- **Task 5 (Suboptimal Search)**: Using BFS for **ClosestDotSearchAgent** was straightforward, but understanding why greedy choices led to suboptimal paths required analyzing maze layouts.